```
#%% md
# 20 aprile



#%%

#%%
l1 = [1, 2, 3]
l2 = [1, 2, 3]
l1_alias = l1

print(hex(id(l1)))
print(hex(id(l1_alias)))
print(hex(id(l2)))
#%%

#%%
l1[0] =100
l1[2] =200

print(l1)
print(l1_alias)
#%%
print(hex(id(l1))) #0x1061bd8c0
print(hex(id(l1_alias))) #0x1061bd8c0
#%%
#



l1 = [1, 2, 3]
l1_alias = l1 #alias
print('prima della assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))

l1_alias = [] # altra lista
print('dopo la assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))
#%%

```

```
l1 = [1, 2, 3]
l1_alias = l1 #alias
print('prima della assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))

l1_alias = 10 # altro oggetto in memoria
print('dopo la assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))
#%%

#%%
a = [1, 2, 3]
a_alias = a
b = [1, 2, 3]
# contenuto
print(a == a_alias)
print(a == b)
# identità
print(a is a_alias)
print(a is b)
#%%

#%%

#%%
a = list(range(10))
b = a[2:5]
c = a[2:5]
print(a)
print(b)
print(c)
print(b is c)

a [2] = 100
b [0] = 200
print(a)
print(b)
print(c)
#%%
a = [1,2, ['x','y','z'] ]
b = a [1:3] #[2, ['x', 'y', 'z']]
print(b)
```

```
print(a is b)
a [1] =200
print('dopo il cambiamento 1')
print(a)
print(b)
# ---
a[2][0] = 'W'
print('dopo il cambiamento 2')
print(a)
print(b)
#%%
a = [1,2, ['x','y','z'] ]
b = a [1:3] #[2, ['x', 'y', 'z']]
# ---
a[2] = 'W'
print('dopo il cambiamento 3')
print(a)
print(b)

#%%
# 22.aprile 2026
# ------
#%%
l1 = [1, [1,2,3],2,3,4,5,6,7,8,9,10]
l2 = l1[0:3]
print(l2[1],type(l2[1]))
l2[1].append(999)
print(l1)


#%%

#%%
l_alias = l1 #alias - stesso indirizzo
l2_shallow = l1[:] # copia superficiale - DIVERSO indirizzo
l2_shallo2= l1.copy()


#%%
import copy
l2_shallo2 = copy.copy(l1)

l_new= copy.deepcopy(l1) #completamente separata
```

```python

#%%

#%%
l1=[10, [1,2,3]]
l2 = l1
l3 = l1[:]
l4 = copy.deepcopy(l1)

l1[0] = 99
print('l1',l1) #[99, [1, 2, 3]]
print('l2',l2) #[99, [1, 2, 3]]
print('l3',l3) #[10, [1, 2, 3]]
print('l4',l4) #[10, [1, 2, 3]]



#%%


l1=[10, [1,2,3]]
l2 = l1
l3 = l1[:]
l4 = copy.deepcopy(l1)

l1[1].append(7)
print(l1) #[10, [1, 2, 3, 7]]
print(l2) #[10, [1, 2, 3, 7]]
print(l3) #[10, [1, 2, 3, 7]]
print(l4) #[10, [1, 2, 3]]
#%%

#%%

#%% md
# Ricapitolando:
#%%
# Ricapitolando

Lista1 = [1, 2, 3]
Lista2 = Lista1[:]   # shallow copy

print(id(Lista1))
print(id(Lista2))
```

```

print(Lista1 is Lista2)  # verifica identità

#id(Lista1) e id(Lista2) sono diversi → oggetti distinti
# Lista1 is Lista2 restituisce False → non sono lo stesso oggetto

# Quindi la shallow copy crea una nuova lista,
# anche se inizialmente contiene gli stessi elementi.
#%%
# la shallow copy crea due liste distinte,
# ma condivide i riferimenti agli stessi oggetti interni.
# Quindi le liste sono indipendenti solo al primo livello

Lista1 = [[1, 2], [3, 4]]
Lista2 = Lista1[:]   # shallow copy

# Verifica identità delle liste
print(Lista1 is Lista2)          # False

# Verifica identità degli elementi interni
print(Lista1[0] is Lista2[0])    # True

# Modifica interna
Lista2[0][0] = 99

print(Lista1)  # cambia anche Lista1
print(Lista2)
# Le liste esterne sono oggetti diversi
# (is → False)
# Gli elementi interni sono gli stessi oggetti
# # (is → True)
# Modificare un oggetto mutabile interno (come una lista) si riflette in entrambe
#%% md
Una *shallow copy* in Python si comporta sempre allo stesso modo: crea una nuova lista ma copia solo i riferimenti agli oggetti contenuti, senza duplicarli, indipendentemente dal fatto che siano mutabili o immutabili. La differenza osservabile emerge perché gli oggetti mutabili possono essere modificati *in place*: in questo caso, la modifica è visibile attraverso entrambe le liste che condividono lo stesso riferimento. Gli oggetti immutabili (come tuple o stringhe), pur essendo anch'essi
```

```
condivisi, non possono essere modificati; ogni cambiamento comporta la creazione di un nuovo oggetto, quindi non produce effetti sull’altra lista.

#%% md
aa
aa



a
#%%

#%%

```